Performance of Matrix Transpose with
1. only row-major layout
2. separate row and column-major layouts

Sparsh Mittal

Matrix transpose operation

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Input

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$$

Output

$$A = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix}_{2 \times 3} \qquad A^T = \begin{bmatrix} a & d \\ b & e \\ c & f \end{bmatrix}_{3 \times 2}$$

Consider code for matrix-transpose

- Here is a simple code to show the matrix-transpose:

```
for (i = 0; i < N; i++)
 for (j = 0; j < N; j++)
  B[j][i] = A[i][j];
```

- Assume that both the input and output matrices are stored in the row-major order.
- In above code, we are accessing multiple arrays such that some arrays are accessed by rows and some by columns.
- Merely doing loop-interchange will not help

Observations

- Assume each element, say A[i][j] takes 1B in memory
- Let N=4.
- We are accessing 16 elements of A and 16 elements of B
- There is no reuse of any element of A or B. Each element is accessed (read/written) only once.
- If cache block size is just 1B, there will be 32 misses.
- If cache block size is increased, there is a possibility of reducing the cache misses.
- Assume N=4. Cache-block size =2B. Each element is 1B. We have an 8B fully-associative cache. It can keep 4 blocks

Without cache blocking
A in Row-major
B in <u>column-major</u>

$A_{0,0}$	$A_{0,1}$	$A_{0,2}$	$A_{0,3}$
$A_{1,0}$	$A_{1,1}$	$A_{1,2}$	$A_{1,3}$
$A_{2,0}$	$A_{2,1}$	$A_{2,2}$	$A_{2,3}$
$A_{3,0}$	$A_{3,1}$	$A_{3,2}$	$A_{3,3}$

$B_{0,0}$	$B_{0,1}$	$B_{0,2}$	$B_{0,3}$
$B_{1,0}$	$B_{1,1}$	$B_{1,2}$	$B_{1,3}$
$B_{2,0}$	$B_{2,1}$	$B_{2,2}$	$B_{2,3}$
$B_{3,0}$	$B_{3,1}$	$B_{3,2}$	$B_{3,3}$

Row -major

$A_{0,0}$	$A_{0,1}$	$A_{0,2}$	$A_{0,3}$	$A_{1,0}$	$A_{1,1}$	$A_{1,2}$	$A_{1,3}$	$A_{2,0}$	$A_{2,1}$	$A_{2,2}$	$A_{2,3}$	$A_{3,0}$	$A_{3,1}$	$A_{3,2}$	$A_{3,3}$

Column -major

$B_{0,0}$	$B_{1,0}$	$B_{2,0}$	$B_{3,0}$	$B_{0,1}$	$B_{1,1}$	$B_{2,1}$	$B_{3,1}$	$B_{0,2}$	$B_{1,2}$	$B_{2,2}$	$B_{3,2}$	$B_{0,3}$	$B_{1,3}$	$B_{2,3}$	$B_{3,3}$

Without cache-blocking, B in column-major

B00	B01	B02	B03
B10	B11	B12	B13
B20	B21	B22	B23
B30	B21	B22	B23

Matrix B

A00	A01	A02	A03
A10	A11	A12	A13
A20	A21	A22	A23
A30	A21	A22	A23

Matrix A

```
for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
         B[j][i] = A[i][j];
```

Let us show indexes as (i,j)

Iteration 1. (0,0). We need A00 and B00. We access one block each from A and B

Cache	A00 A01	B00 **B10**			2 miss

Iteration 2. (0,1). Need A01 and B10. Both are cache hits

Cache	A00 A01	B00 B10			2Hits

Iteration 3. (0,2). Need A02 and B20. Both are cache miss. Fetch them.

Cache	A00 A01	B00 B10	B20 B30	A02 A03	2 miss

Iteration 4. (0,3). Need A03 and B30. Both are hits

Cache	A00 A01	B00 B10	B20 B30	A02 A03	2 Hits

Iteration 5. (1,0). Need A10 and B01. Both are miss. Fetch them.

Remove two oldest blocks

Cache	A10 A11	B01 B11	B20 B30	A02 A03	2 miss

Observation 1: In 4 iterations, we get 4 hits and 4 misses. Out of 4 accesses to A, 2 are misses. Same for B.

For whole matrix-transpose, we will get 16 misses and 16 hits

Without cache blocking
A in Row-major
B in Row-major

$A_{0,0}$	$A_{0,1}$	$A_{0,2}$	$A_{0,3}$
$A_{1,0}$	$A_{1,1}$	$A_{1,2}$	$A_{1,3}$
$A_{2,0}$	$A_{2,1}$	$A_{2,2}$	$A_{2,3}$
$A_{3,0}$	$A_{3,1}$	$A_{3,2}$	$A_{3,3}$

$B_{0,0}$	$B_{0,1}$	$B_{0,2}$	$B_{0,3}$
$B_{1,0}$	$B_{1,1}$	$B_{1,2}$	$B_{1,3}$
$B_{2,0}$	$B_{2,1}$	$B_{2,2}$	$B_{2,3}$
$B_{3,0}$	$B_{3,1}$	$B_{3,2}$	$B_{3,3}$

Row -major

$A_{0,0}$	$A_{0,1}$	$A_{0,2}$	$A_{0,3}$	$A_{1,0}$	$A_{1,1}$	$A_{1,2}$	$A_{1,3}$	$A_{2,0}$	$A_{2,1}$	$A_{2,2}$	$A_{2,3}$	$A_{3,0}$	$A_{3,1}$	$A_{3,2}$	$A_{3,3}$

Row -major

$B_{0,0}$	$B_{0,1}$	$B_{0,2}$	$B_{0,3}$	$B_{1,0}$	$B_{1,1}$	$B_{1,2}$	$B_{1,3}$	$B_{2,0}$	$B_{2,1}$	$B_{2,2}$	$B_{2,3}$	$B_{3,0}$	$B_{3,1}$	$B_{3,2}$	$B_{3,3}$

Without cache-blocking, B in row-major

B00	B01	B02	B03
B10	B11	B12	B13
B20	B21	B22	B23
B30	B21	B22	B23

A00	A01	A02	A03
A10	A11	A12	A13
A20	A21	A22	A23
A30	A21	A22	A23

```
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++)
        B[j][i] = A[i][j];
```

Let us show indexes as (i,j)

Iteration 1. (0,0). We need A00 and B00. We access one block each from A and B

Cache	A00 A01	B00 **B01**			2 miss

Iteration 2. (0,1). Need A01 and B10. A01 is cache-hit. B10 is cache miss

Cache	A00 A01	B00 B01	B10 B11		1Hit 1Miss

Iteration 3. (0,2). Need A02 and B20. Both are cache miss. Fetch them.
Remove B00 B01 block, which is oldest

Cache	A00 A01	B20 B21	B10 B11	A02 A03	2 miss

Iteration 4. (0,3). Need A03 and B30. A03 is hit. B30 is miss. Fetch it.
Remove A00 A01 block, which is oldest

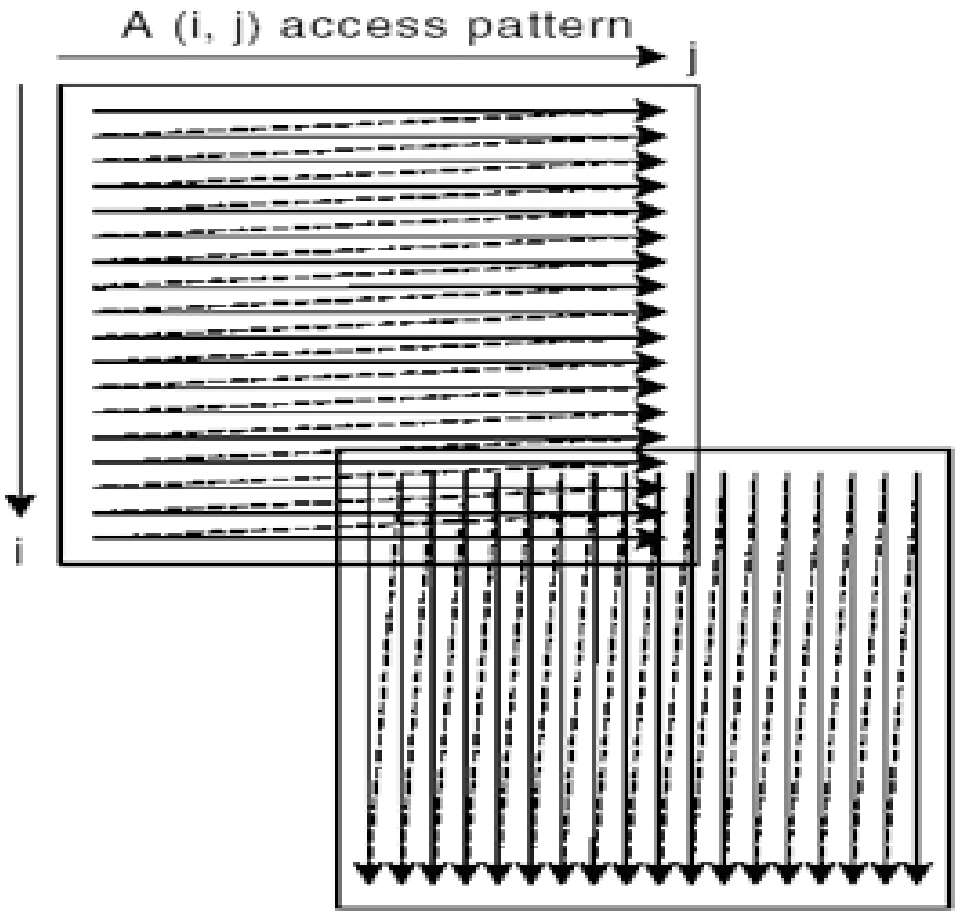
Cache	B30 B31	B20 B21	B10 B11	A02 A03	1 Hit 1 miss

Iteration 5. (1,0). Need A10 and B01. Both are miss. Fetch them.
Remove oldest block

Cache	B30 B31	B20 B21	A10 A11	B00 B01	2 miss

Observation 1: In 4 iterations, we get 2 hits and 6 misses. All accesses to B are misses. Out of 4 accesses to A, 2 are misses.
For whole matrix-transpose, we will get 16+8 misses and 8 hits

Observations



B(j,i) access pattern

Matrix-transpose accesses A in row-major order and B in column-major order

By storing A in row-major order and B in column-major order in memory, then, we can exploit the spatial locality. But this is not possible.

- Observation 2: We had fetched B00 B01 block earlier but later evicted it. We are now fetching it again. How to avoid this waste.
- Answer: cache blocking.

- Changing the layout of B from row-major to column-major will require changes in **hardware** (memory).
- Doing cache-blocking will require changes in **software** (code).